"Año de la Esperanza y el Fortalecimiento de la Democracia"

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

(Universidad del Perú, DECANA DE AMÉRICA)

FACULTAD DE INGENIERÍA ELECTRÓNICA Y ELÉCTRICA

ESCUELA PROFESIONAL DE INGENIERÍA DE TELECOMUNICACIONES

DOCENTE:

Villafuerte Barreto, Hernan Oswaldo

CURSO:

CIRCUITOS DIGITALES

TEMA:

DISEÑO E IMPLEMENTACIÓN DE UN BUFFER CIRCULAR CON DECODIFICADOR DE DIRECCIONES PARA TRANSMISIÓN/RECEPCIÓN UART EN ARDUINO

INTEGRANTES:

Cueto Salazar, Karina Milagros

Marcas Guerrero Renzo Aldair

Mezarina Loza, Ariana Angeles

Ramirez Ramirez Angie Nicole

Lima-Perú

2026

DISEÑO E IMPLEMENTACIÓN DE UN BUFFER CIRCULAR CON DECODIFICADOR DE DIRECCIONES PARA TRANSMISIÓN/RECEPCIÓN UART EN ARDUINO

I. DEFINICIÓN DEL PROYECTO

En sistemas de telemetría y nodos sensoriales de bajo costo, es común utilizar microcontroladores como Arduino para recibir datos por puerto serie (UART) desde sensores remotos. Sin embargo, cuando el flujo de datos entrantes es más rápido que la capacidad de procesamiento o reenvío del Arduino, se produce pérdida de información. Una solución típica es usar un buffer circular implementado como un arreglo en RAM, pero para cumplir con los conceptos de diseño digital (decodificador de direcciones, memoria direccionable y máquina de estados), se debe hacer explícita la lógica de selección de posición y control de flujo.

El presente proyecto implementa un buffer circular de 16×8 bits utilizando un arreglo en la RAM del Arduino, controlado por una FSM explícita (implementada con switch-case o enum) y un decodificador de direcciones combinacional (función que convierte un contador en índice de memoria). Esto resuelve la pérdida de datos y demuestra conceptos de sistemas digitales en un microcontrolador.

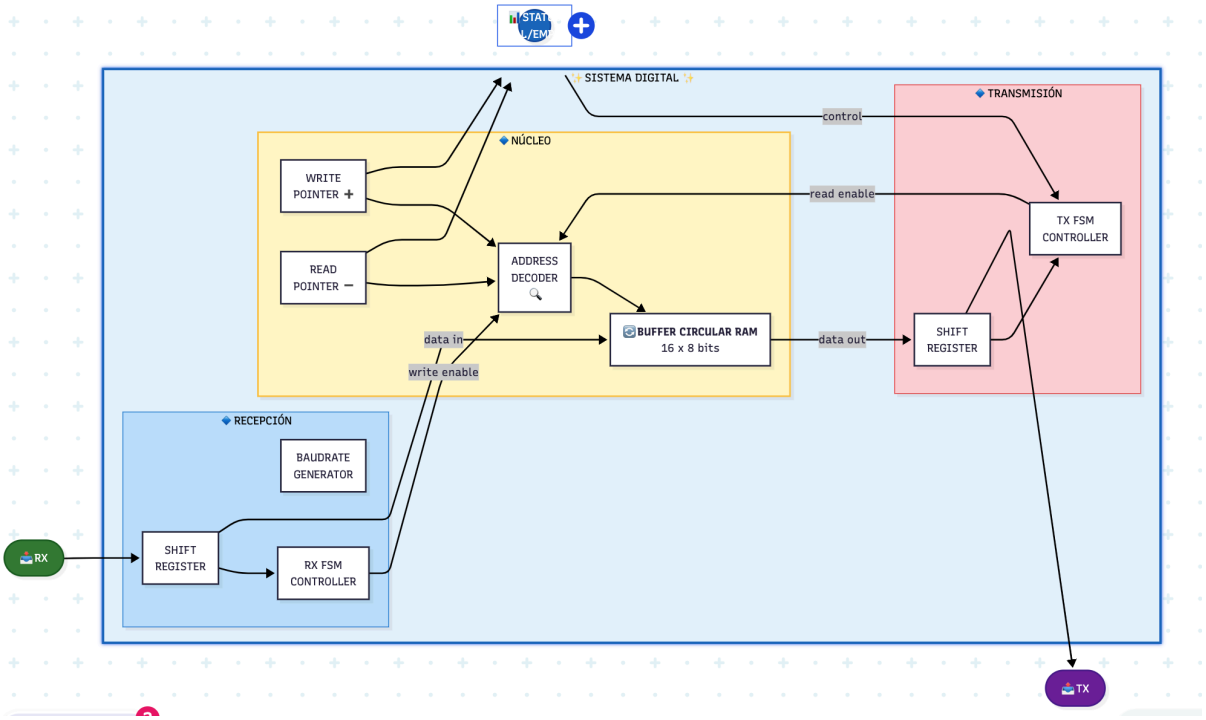
II. OBJETIVOS

- **Objetivo general:**
  Diseñar e implementar en Arduino un sistema digital funcional que integre un buffer circular direccionable para almacenamiento temporal de datos UART, evidenciando explícitamente una FSM, un decodificador de direcciones y una memoria RAM.
- **Objetivos específicos:**

1. Implementar una FSM explícita en C++ que controle los estados de espera, escritura y lectura del buffer circular.
2. Diseñar un decodificador de direcciones combinacional (función addr_decoder()) que convierta un contador de 4 bits en la selección de una posición dentro de un arreglo de 16 bytes.

3. Construir un buffer circular en RAM usando un array `uint8_t buffer[16]` accedido exclusivamente a través del decodificador, no por índice directo.
4. Medir al menos tres métricas: tasa de bits máxima sin pérdida, latencia recepción → eco, y tiempo de ejecución del decodificador (carga en CPU).

III. DIAGRAMA DE BLOQUES



IV. PLATAFORMA SELECCIONADA

Plataforma: Arduino Uno (ATmega328P)

En aplicaciones de telemetría y sensores remotos, los datos recibidos por puerto serie (UART) pueden llegar más rápido de lo que el sistema puede procesarlos o reenviarlos, provocando pérdida de información crítica. Este problema es común en nodos de bajo costo como Arduino, donde no se implementan mecanismos de buffer eficientes. El presente proyecto aborda esta limitación mediante el diseño de un buffer circular direccionable, que permite almacenar hasta 16 bytes de forma temporal mientras el sistema se prepara para procesarlos o retransmitirlos, evitando así la pérdida de datos en ráfagas cortas de comunicación.

Se selecciona la plataforma Arduino Uno por su disponibilidad, bajo costo y capacidad de implementar explícitamente todos los componentes digitales del diseño. A diferencia de una FPGA, Arduino permite una curva de aprendizaje más suave y sigue estando permitido en el enunciado del trabajo académico. Para evaluar el

desempeño del sistema, se medirán tres métricas cuantitativas: tasa de bits máxima sin pérdida de datos, latencia entre la recepción y el eco del byte, y tiempo de ejecución del decodificador como indicador de carga computacional. Estas métricas se obtendrán mediante monitor serie, osciloscopio y la función micros() de Arduino.

Este proyecto no solo resuelve un problema práctico de telecomunicaciones, sino que también demuestra la aplicación integradora de los conceptos de lógica combinacional y secuencial vistos en el curso. La inclusión explícita del decodificador de direcciones y la memoria RAM como parte activa del flujo de datos refleja directamente lo establecido en la sumilla: *"circuitos de decodificación de memorias"* y *"unidades y subsistemas combinacionales y secuenciales"*. El resultado final será un sistema funcional, documentado en un paper formato IEEE, que puede servir como base para futuros trabajos de investigación o aplicaciones en nodos sensoriales de bajo costo.

Requisito de la sumilla	***Implementación en Arduino***	***Evidencia***
Decodificador de direcciones	Función uint8_t addr_decoder(uint8_t counter) que mapea un contador de 4 bits a un índice de 0 a 15. Es combinacional pura (no usa variables de estado, solo lógica de mapeo).	Se accede a la memoria siempre mediante buffer[addr_decoder(contador)], nunca por índice directo.
Memoria (RAM / buffer)	Arreglo uint8_t buffer[16] ubicado en la SRAM del ATmega328P.	Se documenta el mapa de memoria y se demuestra que el decodificador selecciona la posición.
Máquina de estados finitos (FSM)	enum State { IDLE, WRITE, READ, ECHO } + switch(currentState) dentro de loop().	Diagrama de estados en el diseño detallado (E2). La FSM controla los incrementos de los contadores.

Lógica secuencial (contadores)	Variables wr_counter y rd_counter (uint8_t) que se incrementan módulo 16 bajo control de la FSM.	Simulan registros secuenciales que generan la dirección para el decodificador.
Aplicación en telecomunicaciones	Buffer circular para evitar pérdida de datos en recepción UART, típico en telemetría y sensores remotos.	Se demuestra con envío de ráfagas de datos desde un PC y verificación de integridad.

V. MÉTRICAS PROPUESTAS

- **Métrica 1 – Tasa de bits máxima sin pérdida de datos**

Esta métrica cuantifica la máxima velocidad de comunicación en baudios (bits por segundo) que el sistema puede soportar sin que el buffer circular se desborde y se pierdan bytes. Para medirla se enviarán ráfagas de 200 bytes desde un PC al Arduino a velocidades crecientes: 9600, 19200, 38400, 57600 y 115200 bps. Se verificará que todos los bytes enviados sean correctamente almacenados en la RAM mediante el decodificador y luego reenviados (eco). La tasa máxima será aquella en la que se reciba el 100% de los datos sin error. Se utilizará el monitor serie con un script en Python que calcule la integridad de cada trama. Este valor es crítico para aplicaciones de telemetría donde se requiere capturar datos sin pérdidas.

- **Métrica 2 – Latencia entre recepción y eco (microsegundos)**

La latencia mide el tiempo que transcurre desde que el último bit de stop de un byte es recibido por el UART del Arduino, hasta que el primer bit de start del mismo byte (reenviado desde el buffer) aparece en la línea TX. Este tiempo incluye: la interrupción de recepción, la ejecución de la FSM, la llamada al decodificador de direcciones, la escritura en la RAM, la lectura posterior desde otra dirección y la transmisión. Para medirla se conectará un pin digital del Arduino que se ponga a nivel ALTO al inicio de la recepción (por ejemplo, en el ISR(USART_RX_vect)) y a BAJO inmediatamente después de enviar el byte por TX. Usando un osciloscopio se mide la

diferencia de tiempo entre flancos. Se tomarán 100 muestras y se calculará el promedio y la desviación estándar. Esta métrica es clave para sistemas de control en tiempo real.

- **Métrica 3 – Tiempo de ejecución del decodificador de direcciones (ciclos de CPU)**

  El decodificador de direcciones se implementa como una función combinacional (en software) que convierte un contador de 4 bits en un índice de 0 a 15 para acceder al arreglo buffer[16]. Para evaluar su costo computacional se medirá el tiempo que tarda dicha función en ejecutarse, en microsegundos o ciclos de reloj del ATmega328P (16 MHz). Se usará la función micros() antes y después de llamar al decodificador dentro del loop(), realizando 1000 llamadas consecutivas y promediando. Se comparará con el acceso directo por índice (buffer[contador]) para cuantificar la penalización por emular el decodificador. Esta métrica demuestra la diferencia entre una implementación puramente digital (circuito combinacional rápido) y una simulación en software, y es útil para justificar el uso de FPGA en aplicaciones de alta velocidad.

VI. PLAN DE TRABAJO

ACTIVIDAD
Definición de la FSM y simulación conceptual en papel
Implementación del buffer circular con decodificador (función) en Arduino
Implementación de la FSM explícita (switch-case) para control de flujo
E2 – Diseño detallado + simulación funcional (demostración en laboratorio con Serial Monitor)
Integración y pruebas básicas de escritura/lectura en el buffer
Medición de métrica preliminar (recursos de memoria)
E3 – Prototipo hardware inicial y primeras métricas (video de 1 min)

Medición de latencia (osciloscopio) y tasa de bits máxima
E4 – Sistema completo verificado + borrador de paper
Correcciones del paper y preparación del video final
E5 – Paper final IEEE + demostración definitiva

VII. CRONOGRAMA INTERNO

Actividad	Responsable	Entregable asociado
Lectura del enunciado, asignación de roles, definición de la solución conceptual	Todo el equipo	-
Redacción de la propuesta (E1): problema, objetivos, diagrama de bloques, métricas, referencias	Estudiante 1 (líder)	E1 – Propuesta de proyecto
Implementación del decodificador de direcciones y la memoria RAM (buffer) en Arduino	Estudiante 2	-
Implementación de la FSM básica (sin UART aún) y simulación con casos de prueba simples	Estudiante 3	-
Integración del módulo UART (Serial) con la FSM y el buffer. Simulación funcional completa. Preparación de la demostración en laboratorio	Todo el equipo	E2 – Diseño detallado + simulación funcional
Pruebas de escritura/lectura en el buffer. Verificación del decodificador (acceso exclusivo a través de la función).	Estudiante 2 y 3	-

Correcciones		
Medición de métrica preliminar (tiempo de ejecución del decodificador con micros()). Documentación del prototipo	Estudiante 1	-
Grabación del video de 1 minuto mostrando operación básica. Medición de al menos 1 métrica real (tasa de bits o latencia). Preparación del informe E3	Todo el equipo	E3 – Prototipo hardware inicial y primeras métricas
Medición completa de las 3 métricas. Comparación con valores simulados. Análisis de desviaciones	Estudiante 1 y 2	-
Redacción del borrador del paper IEEE (estructura completa). Preparación del video de 5 minutos. Documento E4	Todo el equipo	E4 – Sistema completo verificado + borrador de paper
Revisión del borrador por pares. Correcciones de estilo, formato IEEE y contenido técnico. Ajustes finales del código	Estudiante 3	-
Finalización del paper IEEE (8-15 páginas). Grabación del video final de 8 minutos. Preparación de presentación oral (10 minutos). Entrega E5	Todo el equipo	E5 – Paper final IEEE + demostración definitiva